Process & Decision Documentation

GBDA 302: Side Quest 5

Student Name: Min Htet Naing, dmnaing (21008098)

Project/Assignment Decisions

Description

For Side Quest 5, I built a reflective, side-scrolling experience using the modular Example 5 system. The world is larger than the screen, and the smooth camera (lerp-based) creates calm horizontal motion. Instead of changing the player's physics to express emotion, I used environmental variables such as rain intensity and background colour to create a melancholic mood.

Small green "energy" symbols were placed across each screen section of the world. When the player reaches one, the world briefly becomes lighter and the blob smiles before returning to a sad state. The intention was to explore emotional pacing through subtle environmental feedback.

Key Decision/Change

A key design decision was to preserve the original Example 5 physics and movement system. Rather than slowing the blob or modifying jump mechanics, I chose to express sadness and hope through environmental shifts. This kept the gameplay feel consistent while allowing the camera and world state to communicate emotion.

Another important decision was to make the emotional change temporary (3 seconds). This reinforces the idea that moments of relief exist within a larger melancholic atmosphere, aligning with the reflective theme of the assignment.

GenAI Documentation (Used)

Date Used: Feb 10th, 2026

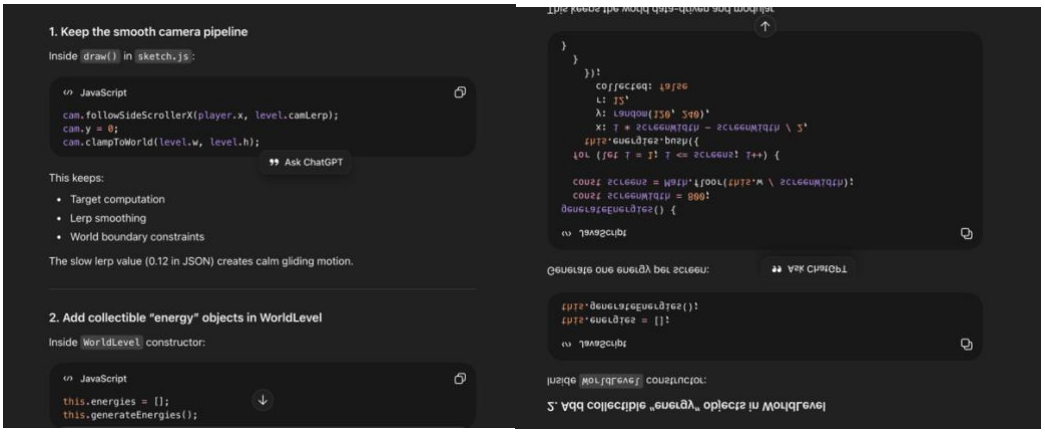
Tool Disclosure: ChatGPT (GPT-5.2 Thinking)

ChatGPT was used to clarify and to confirm that the modification aligned with the Week 5 JSON and class architecture. It also assisted with minor refactoring suggestions for sketch.js and WorldLevel.js. All final code was reviewed, implemented, and tested manually in VS Code and on GitHub Pages.

Appendix

Prompt 1

“How can I create a reflective side-scrolling camera experience in p5.js that uses smooth lerp movement and hides interactive collectible objects across a large JSON-defined world?”



3. Draw glowing green energies

Inside `drawWorld()`:

JavaScript

```javascript
for (let e of this.energies) {
  if (!e.collected) {
    fill(120, 255, 120);
    ellipse(e.x, e.y, e.r * 2);

    fill(120, 255, 120, 80);
    ellipse(e.x, e.y, e.r * 3);
  }
}
```

Ask ChatGPT

4. Detect collision in sketch.js (not in WorldLevel)

Gameplay logic belongs in `sketch.js`, not in the world class.

JavaScript

```javascript
function checkEnergyCollision() {
  for (let e of level.energies) {
    if (!e.collected) {
      let d = dist(player.x, player.y, e.x, e.y);
      if (d < player.r + e.r) {
        e.collected = true;
        player.happyTimer = millis();
      }
    }
  }
}
```

Ask ChatGPT

This preserves modular separation:

- WorldLevel → data + rendering
- BlobPlayer → movement + visuals
- sketch.js → gameplay flow

Prompt 2

“How can I create a reflective side-scrolling camera experience in p5.js that uses smooth lerp movement and hides interactive collectible objects across a large JSON-defined world?”

1. Increase visibility

Inside `drawWorld()`:

JavaScript

```javascript
stroke(160, 200, 255, 180);
strokeWeight(2);
```

2. Increase drop length

JavaScript

```javascript
line(drop.x, drop.y, drop.x, drop.y + 14);
```

3. Add subtle speed variation

In constructor:

JavaScript

```javascript
speed: random(3, 6)
```

4. Optional darker sad background

Inside `drawWorld()` before drawing platforms:

JavaScript

```javascript
background(35, 45, 60);
```

Why This Works

- Only modifies rendering
- Does not touch collision
- Keeps world logic clean
- Enhances emotional tone through atmosphere

Prompt 3

“How do I ensure emotional changes remain temporary and reinforce pacing rather than permanently altering gameplay?”

Ask ChatGPT

1. Store timestamp

JavaScript
```
this.happyTimer = millis();
```

2. Check duration inside draw

JavaScript
```
let isHappy = millis() - this.happyTimer < 3000;
```

3. Do NOT modify:

- gravity
- jumpV
- vx / vy
- collision logic

This ensures emotion is aesthetic only.

Prompt 4

“How do I ensure emotional changes remain temporary and reinforce pacing rather than permanently altering gameplay?”

1. Store emotion-related properties inside BlobPlayer

Inside constructor:

JavaScript
```
this.happyTimer = 0;
this.happyDuration = 3000;
```

2. Add a helper method

JavaScript
```
isHappy() {
  return millis() - this.happyTimer < this.happyDuration;
}
```

3. Only modify visuals inside draw()

JavaScript
```
let happy = this.isHappy();

if (happy) {
  arc(this.x, this.y + 5, 16, 12, 0, PI);
} else {
  arc(this.x, this.y + 10, 16, 12, PI, TWO_PI);
}
```

Why This Matters

- WorldLevel remains environment-only
- Camera2D remains view-only
- sketch.js triggers state change
- BlobPlayer owns emotion rendering

This respects the separation-of-concerns model from Example 5.

Prompt 5

“Where should collectible collision detection live in a modular p5.js architecture?”

Answer:

Collision between player and collectibles is gameplay logic, so it belongs in `sketch.js`.

Ask ChatGPT

ce in:

- WorldLevel (environment only)
- BlobPlayer (movement only)

2. Add inside draw()

JavaScript

```
function checkEnergyCollision() {
  for (let e of level.energies) {
    if (!e.collected) {
      let d = dist(player.x, player.y, e.x, e.y);
      if (d < player.r + e.r) {
        e.collected = true;
        player.happyTimer = millis();
      }
    }
  }
}
```

```
  }
}
```

Call it after player.update(level):

JavaScript

```
checkEnergyCollision();
```

Architecture Preserved

- JSON → data
- WorldLevel → render geometry + rain
- BlobPlayer → physics
- sketch.js → game rules

Prompt 6

"How can I use lerp to create a calmer, more reflective camera movement?"

Answer:

The camera smoothness comes from a small lerp value stored in JSON.

1. In levels.json

JSON

```
"camera": { "lerp": 0.12 }
```

Lower value = smoother and slower glide.

2. In draw()

JavaScript

2. In draw()

JavaScript

```
cam.followSideScrollerX(player.x, level.camLerp);
cam.clampToWorld(level.w, level.h);
```

Why It Feels Reflective

- Camera lags slightly behind player
- Motion feels weighted
- World reveals gradually
- Encourages slower perception

This creates emotional pacing without cha ng physics.

Prompt 7

"How can I visually communicate sadness using environmental design instead of modifying character physics?"

Answer:

Emotion can be conveyed through color, motion, and lighting.

1. Dark muted background

JavaScript

```
background(35, 45, 60);
```

2. Visible rain

JavaScript

```
stroke(160, 200, 255, 180);
strokeWeight(2);
line(drop.x, drop.y, drop.x, drop.y + 14);
```

3. Desaturated blob color

In levels.json:

JSON

```
"blob": "#7E8DAA"
```

Result

The gameplay remains identical, but:

- Color palette communicates mood
- Rain adds atmosphere
- Camera smoothness reinforces tone

This aligns with the Week 5 goal: pacing + motion evoke emotion.

Prompt 8

“How can I ensure the world is larger than the screen while maintaining correct camera clamping?”

Answer:

Ask ChatGPT e world in JSON

JSON

```
"world": { "w": 2400, "h": 360 }
```

2. Clamp camera inside draw()

JavaScript

```
cam.clampToWorld(level.w, level.h);
```

3. Ensure player stays inside world

Inside BlobPlayer.update():

JavaScript

```
this.x = constrain(this.x, this.r, level.w - this.r);
```